DL-06 Plan: Persistence, Active Learning, Scheduling + Stats

Context

DL-05 closed the HITL loop but left a critical gap: approvals and agent_log live in memory only. Server restart = lost queue. Jenny can't trust a pending $11k PO to survive. DL-06 closes this with SQLite, adds active learning (EMA supplier scores), schedules Betsy to run autonomously, and adds a stats panel to betsy.html. Together these turn Betsy from a tool you trigger into a system that runs itself and gets smarter.

Four pillars and build order

Dependencies run top to bottom -- each step is independently testable.

1. `server/db.py` (new file) -- SQLite persistence

Built-in `sqlite3`, zero new dependencies.

Schema:

```
CREATE TABLE IF NOT EXISTS agent_log (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp  TEXT NOT NULL,
    trigger    TEXT NOT NULL,
    analysis   TEXT NOT NULL,
    decision   TEXT NOT NULL,
    confidence REAL NOT NULL DEFAULT 0.0,
    metadata   TEXT NOT NULL DEFAULT '{}'   -- JSON blob
);

CREATE TABLE IF NOT EXISTS approvals (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    decision_id TEXT UNIQUE NOT NULL,
    status      TEXT NOT NULL DEFAULT 'pending',
    action      TEXT NOT NULL,
    sku_id      TEXT,
    supplier_id TEXT,
    po_total    REAL,
    qty         INTEGER,
    unit_price  REAL,
    confidence  REAL NOT NULL DEFAULT 0.5,
    reasoning   TEXT,
    payload     TEXT,           -- JSON blob
    created_at  TEXT NOT NULL,
    resolved_at TEXT
);
```

Functions:

```
init_db()
save_log_entry(entry: dict)
load_log_entries() -> list
clear_log()
```

```
save_approval(item: dict)
update_approval(decision_id, status, resolved_at)
load_all_approvals() -> list
```

Write lock: module-level `threading.Lock()` around every write. One DB file: `betsy.db` at project root.

Supplier scores are NOT persisted -- they are session-only (in-memory from mock JSON). Reason: `state.reset()` reloads suppliers for scenario testing. Score learning is demonstrated within a session, which is enough for portfolio evidence.

---

2. Wire state.py and routers to db.py

`server/state.py` -- in `AppState.__init__`, after `self.load()`:

```
from server import db
db.init_db()
self.agent_log = db.load_log_entries()
self.approvals = db.load_all_approvals()
```

In `reset()` -- keep `self.agent_log = []` AND call `db.clear_log()`. Don't touch approvals (DL-05 lesson).

`server/routers/agent_log.py` -- in `add_log_entry()`, after `state.agent_log.append(record)`:

```
db.save_log_entry(record)
```

In `clear_log()`, also call `db.clear_log()`.

`server/routers/approvals.py` -- in `queue_approval()`, after `state.approvals.append(item)`:

```
db.save_approval(item)
```

In `approve()` and `reject()`, after updating the in-memory dict:

```
db.update_approval(decision_id, status, resolved_at)
```

Test: start server -> run a scenario -> restart server -> verify log entries and pending approvals still present.

---

3. EMA supplier score updates -- `server/routers/orders.py`

When a PO is marked "delivered", recalculate the supplier's `reliability_score`.

Formula:

```
delivery_performance = 1.0  if on time
                     = max(0.0, 1.0 - lateness_days * 0.1)  if late

new_score = 0.2 * delivery_performance + 0.8 * old_score
new_score = clamp(new_score, 0.0, 1.0)
```

Alpha = 0.2: new delivery counts 20%, existing history 80%. 5 days late -> performance = 0.5. 10+ days late -> 0.0.

PATCH endpoint change (backward-compatible):

```
@router.patch("/{po_id}/status")
def update_order_status(po_id: str, status: str, actual_delivery: str = None):
    ...
    if status == "delivered":
        order["actual_delivery"] = actual_delivery or datetime.now().isoformat()
```

```
            _update_supplier_score(order)
        ...
```

`_update_supplier_score()` logs the before/after to `state.agent_log` with trigger `"ema_score_update"` and metadata `{supplier_id, old_score, new_score, performance, lateness_days}`. This is the evidence for the DL document.

Test: GET /api/suppliers (record baseline) -> PATCH PO to delivered -> GET /api/suppliers (verify score changed). Run twice: once on-time, once with `actual_delivery` set 5 days past `expected_delivery`.

---

4. APScheduler -- `server/main.py` + `server/scheduler_instance.py`

Why a separate module: `stats.py` needs to read the scheduler to get `next_run_time`. Importing `scheduler` directly from `main.py` causes a circular import. Solution: put the scheduler instance in its own tiny module.

`server/scheduler_instance.py` (new, 3 lines):

```
from apscheduler.schedulers.background import BackgroundScheduler
scheduler = BackgroundScheduler()
```

`server/main.py` changes:

```
from contextlib import asynccontextmanager
from server.scheduler_instance import scheduler

@asynccontextmanager
async def lifespan(app: FastAPI):
    from server.db import init_db
    init_db()
    interval = int(os.getenv("AGENT_INTERVAL_MINUTES", "30"))
    scheduler.add_job(_scheduled_run, "interval", minutes=interval,
                      id="betsy_auto_run", replace_existing=True)
    scheduler.start()
    yield
    scheduler.shutdown(wait=False)

def _scheduled_run():
    try:
        from pipeline.run import run_full
        run_full(scenario=None)
    except Exception as exc:
        logging.getLogger("betsy.scheduler").error("Scheduled run failed: %s", exc)

app = FastAPI(title="Betsy Mock Server", lifespan=lifespan, ...)
```

`requirements.txt`: add `apscheduler>=3.10.0`

?? Known limitation to note in DL: If a scheduled run and a manual run overlap, two agents can enqueue approvals simultaneously. Fine for a school project demo, worth noting.

Test: set `AGENT_INTERVAL_MINUTES=1`, start server, wait 90s, check agent_log has a new entry from the auto-run.

---

5. Stats endpoint -- `server/routers/stats.py` **(new file)**

```
GET /api/stats
```

Response:

```
{
  "decisions_total": 42,
  "decisions_auto": 31,
  "decisions_human": 11,
  "auto_rate_pct": 73.8,
  "pending_approvals": 2,
  "queue_value_eur": 14250.00,
  "last_run": "2026-05-31T14:30:00",
  "scheduler_active": true,
  "next_run": "2026-05-31T15:00:00"
}
```

Computed from `state.agent_log` and `state.approvals` (already loaded from DB). Reads `scheduler` from `server/scheduler_instance.py` -- no circular import.

Register in `server/main.py`: `app.include_router(stats.router)`

---

6. betsy.html -- stats panel

Insert above the quick-actions row. 5-column grid, collapses to 2 on mobile.

Metrics: - Decisions made -- `stats.decisions_total` - Autonomous rate -- `stats.auto_rate_pct + '%'` - Awaiting your OK -- `stats.pending_approvals` - Value in queue -- `?stats.queue_value_eur` - Next auto-run -- `timeAgo(stats.next_run) + ' (auto)'`

Add `get('/api/stats').catch(() => null)` to the existing `Promise.all` in `refresh()`. Graceful fallback if stats endpoint isn't up.

---

7. `tests/test_ema_learning.py` -- learning loop evidence

Standalone script (not pytest), requires server running.

1. Record baseline supplier scores
2. Find an in-transit PO -> mark as delivered (on time) -> record new score
3. Create a new PO via POST -> mark as delivered 5 days late -> record new score
4. Print comparison table:

```
Supplier           | Baseline | After on-time | After late (+5d)
-------------------------------------------------------------------
PrecisionParts GmbH | 0.9700  | 0.9760        | 0.8808
```

Formula verification column: `0.2*1.0 + 0.8*0.97 = 0.976` -- explainable, DL-ready.

Critical files

```
File  |  Action
---------------
server/db.py  |  NEW -- full persistence layer
-----------------------------------------
server/scheduler_instance.py  |  NEW -- scheduler singleton
-----------------------------------------------------------
server/routers/stats.py  |  NEW -- stats endpoint
-----------------------------------------------------------
server/state.py  |  MOD -- load from DB on init
```

```
-----------------------------------------------------------
server/main.py  |  MOD -- lifespan, APScheduler, db init
-----------------------------------------------------------
server/routers/agent_log.py  |  MOD -- write to DB
-----------------------------------------------------------
server/routers/approvals.py  |  MOD -- write to DB
-----------------------------------------------------------
server/routers/orders.py  |  MOD -- EMA on delivery
-----------------------------------------------------------
dashboard/betsy.html  |  MOD -- stats panel
-----------------------------------------------------------
requirements.txt  |  MOD -- add apscheduler&gt;=3.10.0
-----------------------------------------------------------
tests/test_ema_learning.py  |  NEW -- EMA evidence script
-----------------------------------------------------------
```

Discussion points before building

1. EMA alpha = 0.2 -- this is moderately aggressive. One late delivery on an otherwise good supplier (0.97) drops them to 0.876 after 5 days late. Is that the right sensitivity, or do you want slower learning (alpha=0.1)?
2. Scheduler default interval -- 30 min is good for production, but for the demo we'll want to set `AGENT_INTERVAL_MINUTES=2` during testing. Should I bake a shorter default in or leave it at 30 and we just use the env var?
3. Stats scope -- the 5 metrics above are the practical minimum. Do you want anything else visible? (e.g. supplier score change history, run timing, error count?)
4. Supplier score history -- right now score changes are only visible in the agent_log entries. If you want a chart or history table in betsy.html it would need a `supplier_scores` table in the DB. Worth it or overkill for the DL?